Algorytmy i struktury danych
Lista 3

Jan Mikołajczyk 287396

19 Stycznia 2026

---

1 Algorytmy CUT ROD

1.1 Rekurencyjne rozwiązanie problemu cięcia pręta

Niech p_j oznacza cenę pręta o długości j, a $R(n)$ maksymalny zysk możliwy do uzyskania z pręta o długości n. Wtedy zachodzi rekurencja:

$$R(0) = 0,$$

$$R(n) = \max_{1 \le j \le n} \left(p_j + R(n-j) \right).$$

Dla każdej możliwej pierwszej długości odciętego fragmentu j algorytm sprawdza, jaki zysk daje sprzedaż kawałka długości j oraz optymalny podział pozostałej części długości $n - j$. Spośród wszystkich możliwości wybierana jest ta, która daje największy łączny dochód.

1.2 Podejście z użyciem memoizacji do algorytmu CUT ROD

```
int CUT_ROD_MEMO(int n, vector<int>& price, vector<int>& memo) {
    if (n==0) return 0;
    if (memo[n]!=-1) return memo[n];
    int answer = 0;
    for (int j=1 ;j<=n; j++) {
        answer = max(answer, price[j-1] + CUT_ROD_MEMO(n-j, price, memo));
    }
    return memo[n] = answer;
}
```

Figure 1: Kod z użyciem memoizacji w celu wykonywania każdego podproblemu tylko raz.

Wersja z memoizacją eliminuje wielokrotne obliczanie tych samych podproblemów, przechowując ich wyniki w tablicy pomocniczej. Dzięki temu, że każdy podproblem jest rozwiązywany tylko raz, a złożoność czasowa algorytmu spada z wykładniczej do $O(n^2)$.

1.3 Podejście iteracyjne do algorytmu CUT ROD

```
int CALL_CUT_ROD_ITER(vector<int>& price) {
    int n = price.size();
    vector<int> tab(n+1, 0);
    for (int i=1 ;i<=n ;i++) {
        int max_val=0;
        for (int j=1;j<=i;j++) {
            max_val = max(max_val, price[j-1] + tab[i-j]);
        }
        tab[i] = max_val;
    }
    return tab[n];
}
```

Figure 2: Iteracyjny algorytm **CUT ROD** wykorzystujący programowanie dynamiczne

W podejściu iteracyjnym rezygnujemy z rekurencji na rzecz dynamicznego programowania typu *bottom–up*. Tworzymy tablicę *tab*, gdzie $tab[i]$ oznacza maksymalny zysk możliwy do uzyskania z pręta o długości i.

Złożoność czasowa tego algorytmu wynosi $O(n^2)$.

Wnioski dotyczące różnych podejść do algorytmy CUT ROD

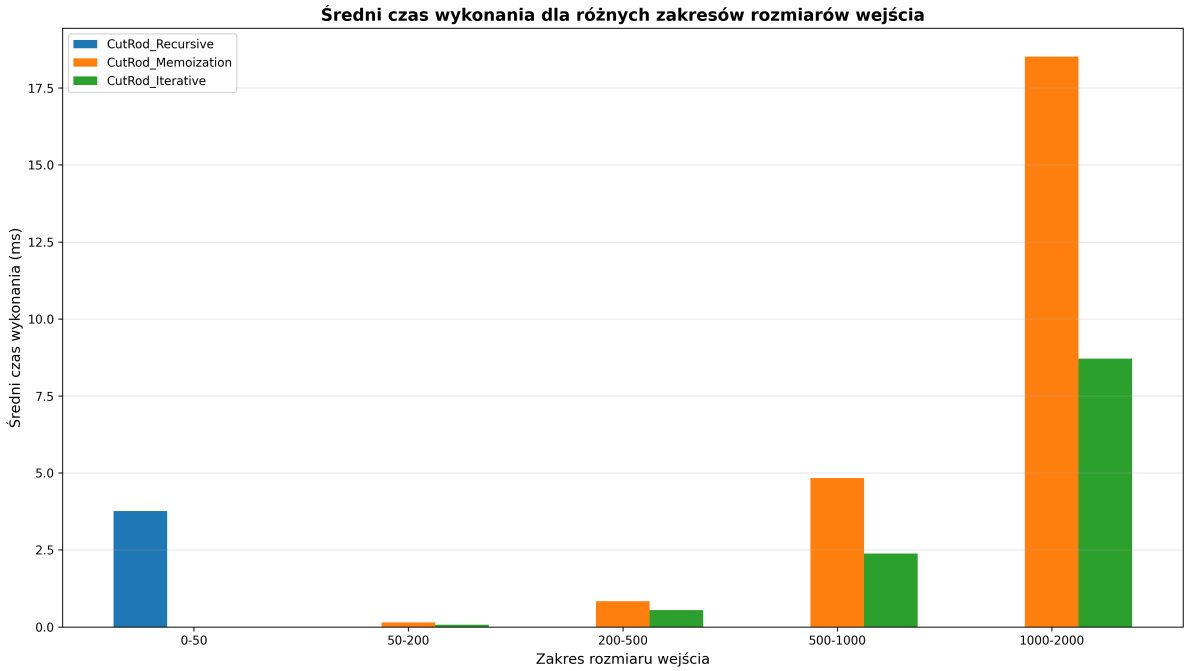


Figure 3: Porównanie każdego podejścia dla różnych wielkości próbek danych

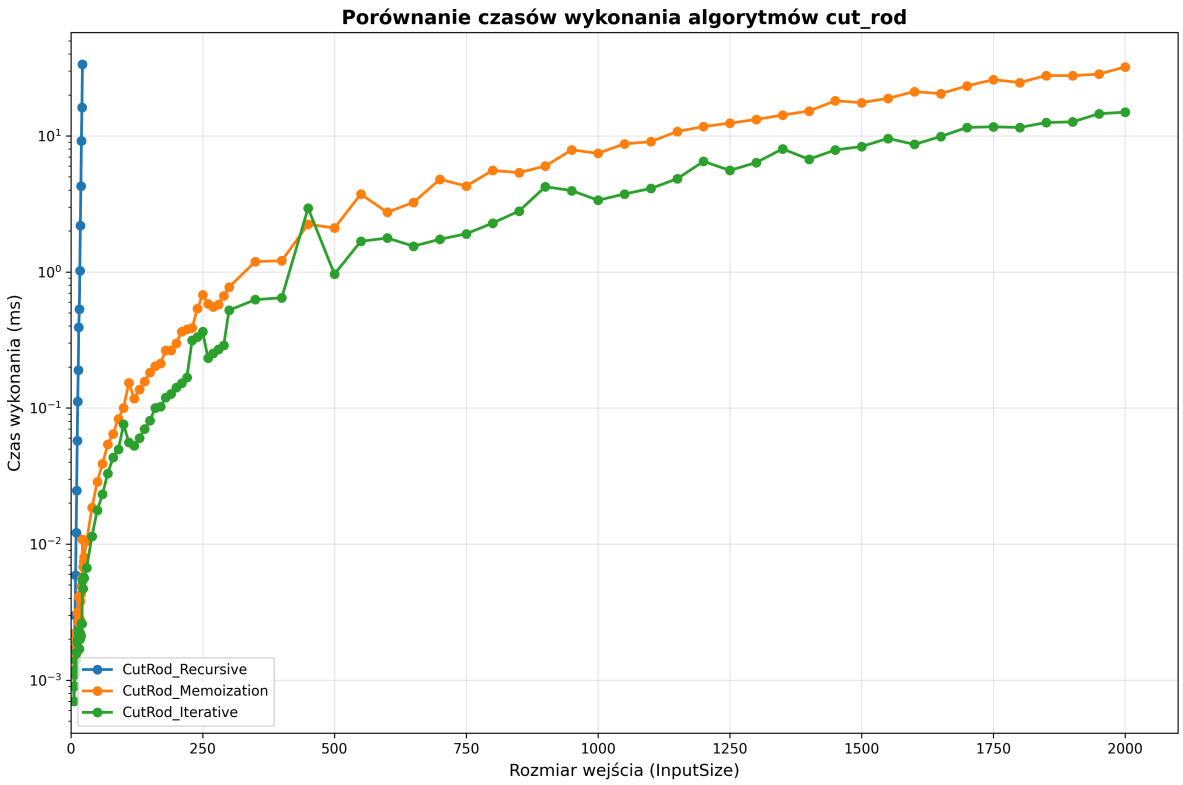


Figure 4: Dokładny stosunek czasu do wielkości danych

1.4 Wnioski z analizy pomiarów i testów

Na podstawie pomiarów czasu wykonania trzech wersji algorytmu cięcia pręta można sformułować następujące wnioski:

- **Algorytm rekurencyjny** bez zapamiętywania wyników skaluje się bardzo słabo. Czas obliczeń rośnie wykładniczo wraz z n, co wynika z wielokrotnego rozwiązywania tych samych podproblemów. Już dla umiarkowanych rozmiarów wejścia staje się on niepraktyczny.
- **Algorytm z memoizacją** znacząco poprawia wydajność dzięki zapisywaniu wyników w tablicy `memo`. Każdy podproblem jest liczony tylko raz, co redukuje złożoność do $O(n^2)$. Dane potwierdzają dobrą skalowalność nawet dla dużych wartości n.
- **Algorytm iteracyjny (bottom–up)** jest najszybszy w praktyce. Brak rekurencji i sekwencyjne wypełnianie tablicy dynamicznej zmniejszają narzut obliczeniowy, przy zachowaniu tej samej złożoności $O(n^2)$.

2 Algorytmy LCS (Longest Common Subsequence)

2.1 Iteracyjne rozwiązanie problemu LCS

```cpp
int CALL_LCS_ITER_SILENT(string& s1, string& s2) {
    int m = s1.size();
    int n = s2.size();
    vector<vector<int>> matrix(m+1, vector<int>(n+1, 0));
    for (int i=1; i<=m; i++) {
        for (int j=1; j<=n; j++) {
            if (s1[i-1]==s2[j-1]) {
                matrix[i][j] = matrix[i-1][j-1]+1;
            } else {
                matrix[i][j] = max(matrix[i-1][j], matrix[i][j-1]);
            }
        }
    }
    return matrix[m][n];
}
```

Figure 5: Algorytm LCS z podejściem iteracyjnym

Algorytm oblicza długość najdłuższego wspólnego podciągu (LCS – *Longest Common Subsequence*) dla dwóch napisów s_1 i s_2 o długościach odpowiednio m i n. Wykorzystuje on dynamiczne programowanie w podejściu bottom–up.

Niezmiennik pętli. Po zakończeniu każdej iteracji pętli zewnętrznej dla danego i oraz każdej iteracji pętli wewnętrznej dla danego j zachodzi:

$$matrix[i][j] = \text{długość LCS dla } s_1[1..i] \text{ i } s_2[1..j].$$

Oznacza to, że w każdej chwili tablica zawiera poprawne rozwiązania wszystkich podproblemów odpowiadających prefiksom krótszym lub równym aktualnie przetwarzanym.

2.2 Porównanie podejścia iteracyjnego z podejściem rekurencyjnym z memoizacją

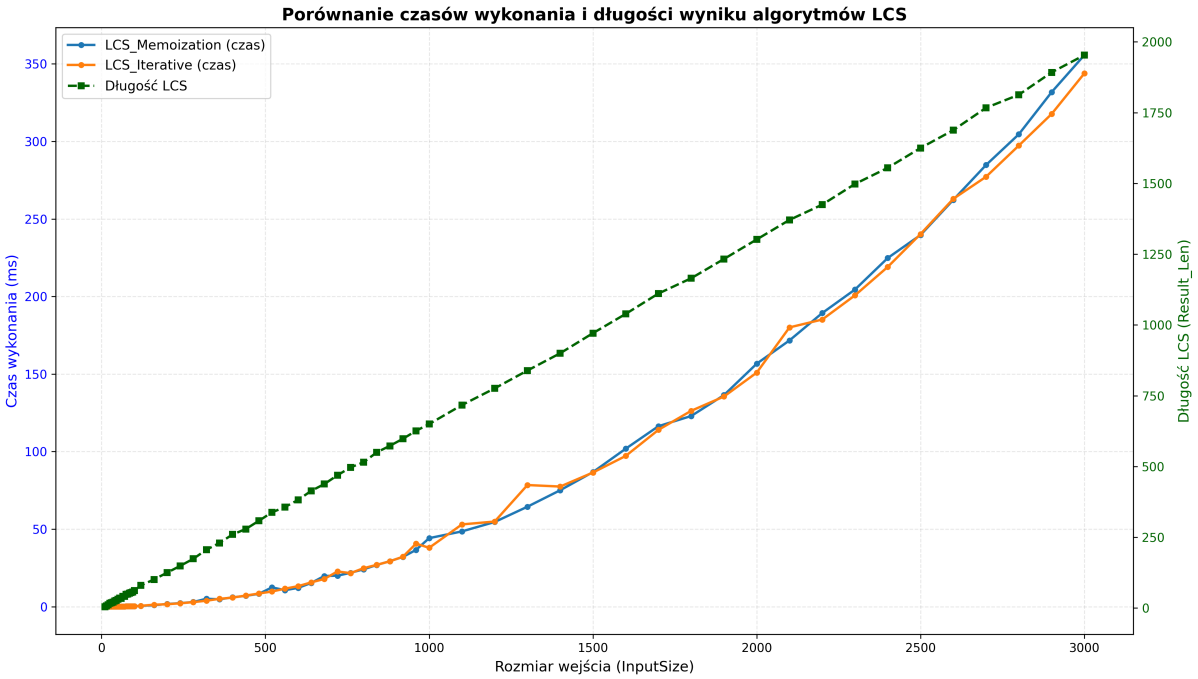


Figure 6: Porównanie obu podejść w kontraście z długością LCS

W badaniu porównano dwie implementacje obliczania długości najdłuższego wspólnego podciągu (LCS) dla napisów s_1 i s_2:

1. **Rekurencja z memoizacją** (`CALL_LCS_RECURENCY_MEMO`) – klasyczna rekurencja, w której wyniki podproblemów są zapisywane w macierzy `memo`. Każdy element `memo[n][m]` jest obliczany tylko raz, a kolejne wywołania korzystają z zapisanej wartości.

2. **Iteracyjne podejście bottom–up** (`CALL_LCS_ITER_SILENT`) – macierz dynamiczna wypełniana wiersz po wierszu, bez wywołań rekurencyjnych.

Analiza teoretyczna: Obie metody tworzą macierz rozmiaru $(n+1) \times (m+1)$ i każdy element liczą dokładnie raz. Z tego względu ich złożoność czasowa wynosi:

$$O(m \cdot n),$$

Wnioski z wykresu:

- Dla małych rozmiarów napisów oba algorytmy osiągają bardzo podobne czasy wykonania.

```cpp
struct Activity {
    int id;
    int start;
    int finish;
};

bool compare_start(Activity a, Activity b) { return a.start < b.start; }
```

Figure 8: Definiujemy pomocniczy typ zmiennych (strukture) i funkcje

- Dla większych danych ($n, m > 1000$) można zaobserwować drobne różnice: czas iteracyjny może być nieco niższy lub wyższy niż wersji rekurencyjnej z memoizacją.

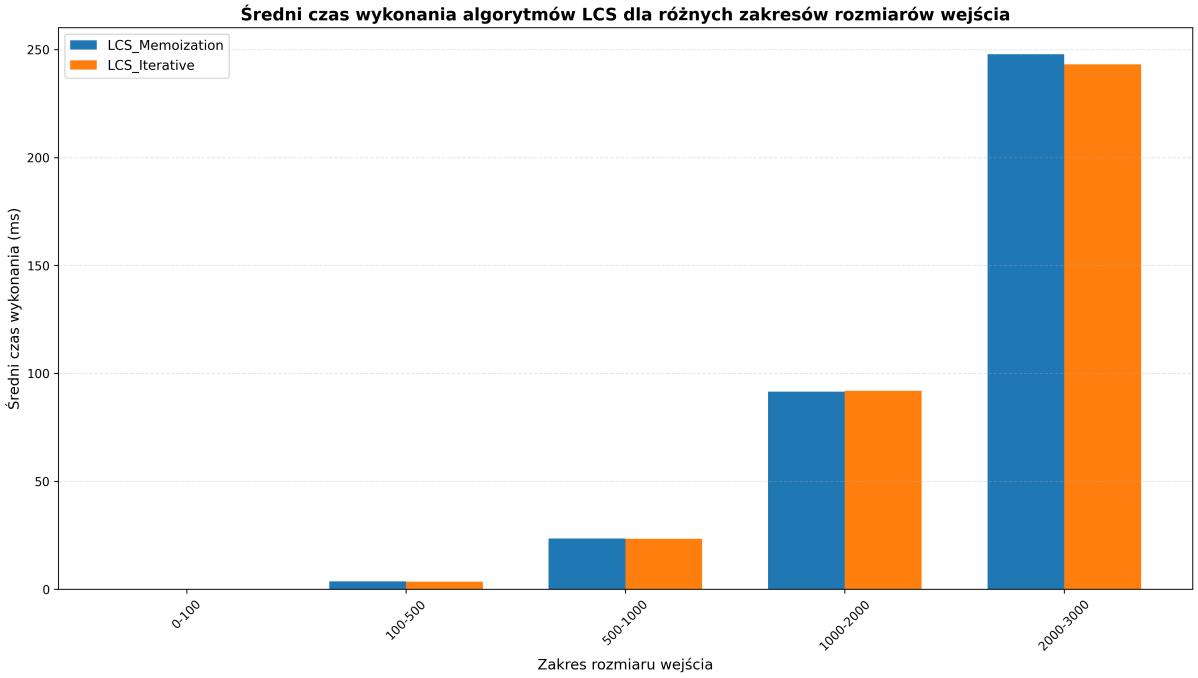


Figure 7: Porównanie słupkowe

3 Activity Selector

Activity Selector polega na wybraniu maksymalnej liczby niepokrywających się aktywności z danego zbioru, gdzie każda aktywność ma określony czas rozpoczęcia i zakończenia.

3.1 Activity Selector podejście rekurencyjne

3.2 Rekurencyjny Activity Selector (od końca)

Przedstawiony algorytm rozwiązuje problem wyboru maksymalnej liczby niepokrywających się aktywności, korzystając z podejścia rekurencyjnego i rozpoczynając wybór od ostatniej aktywności.

```
void RECURENCY_STEP_ACTIVITY_SELECTOR(vector<Activity>& activities, int k, vector<Activity>& result) {
    int m=k-1;
    while (m >= 0 && activities[m].finish > activities[k].start) {
        m--;
    }
    if (m>=0) {
        result.push_back(activities[m]);
        RECURENCY_STEP_ACTIVITY_SELECTOR(activities, m, result);
    }
}

vector<Activity> ACTIVITY_SELECTOR_RECUR(vector<Activity> activities) {
    if (activities.empty()) return {};
    sort(activities.begin(), activities.end(), compare_start);
    vector<Activity> result;
    int n = activities.size();
    result.push_back(activities[n - 1]);
    RECURENCY_STEP_ACTIVITY_SELECTOR(activities, n-1, result);
    reverse(result.begin(), result.end());
    return result;
}
```

Figure 9: Rekurencyjny algorytm Activity selector

Działanie algorytmu:

1. Wybierana jest **ostatnia aktywność** w posortowanej liście (czyli ta o największym czasie rozpoczęcia), i dodawana do wyniku.
2. Następnie wywoływana jest funkcja rekurencyjna `RECURENCY_STEP_ACTIVITY_SELECTOR`, która:
   - przegląda wszystkie aktywności poprzedzające aktualnie wybraną,
   - wybiera ostatnią, która kończy się przed rozpoczęciem aktualnie wybranej,
   - dodaje ją do wyniku i wywołuje się rekurencyjnie dla tej nowo wybranej aktywności.
3. Po zakończeniu rekurencji lista wynikowa jest odwracana, aby zachować kolejność rosnących czasów rozpoczęcia.

3.3 Activity Selector z programowaniem dynamicznym

```
int FIND_NEXT(vector<Activity>& activities, int finish_time) {
    int low = 0, high = activities.size()-1;
    int ans = activities.size();
    while (low<=high){
        int mid = low + (high-low)/2;
        if (activities[mid].start >= finish_time) {
            ans = mid;
            high = mid-1;
        } else {
            low = mid+1;
        }
    }
    return ans;
}
```

Figure 10: Funkcja FIND NEXT. Część kodu reprezentującego podejście dynamiczne do problemu Activity Selector.

Funkcja `FIND_NEXT`**:** - Funkcja ta przyjmuje czas zakończenia aktywności i wyszukuje pierwszą aktywność w posortowanej liście, która zaczyna się "nie wcześniej niż po zakończeniu aktualnej". - W implementacji wykorzystano "wyszukiwanie binarne", dzięki czemu znalezienie następnej kompatybilnej aktywności odbywa się w czasie $O(\log n)$. - Tablica `next_compatible[i]` przechowuje indeks następnej aktywności dla każdej i, co pozwala efektywnie odtwarzać wynik końcowy.

Podsumowanie: Algorytm wykorzystuje programowanie dynamiczne, ponieważ buduje optymalne rozwiązania dla wszystkich podproblemów w tablicy *tab* od końca do początku, a funkcja `FIND_NEXT` umożliwia szybkie znalezienie kompatybilnych aktywności w celu efektywnego uaktualniania wartości *tab*.

3.4 Porównanie podejścia rekurencyjnego, iteracyjnego i z użyciem programowania dynamicznego

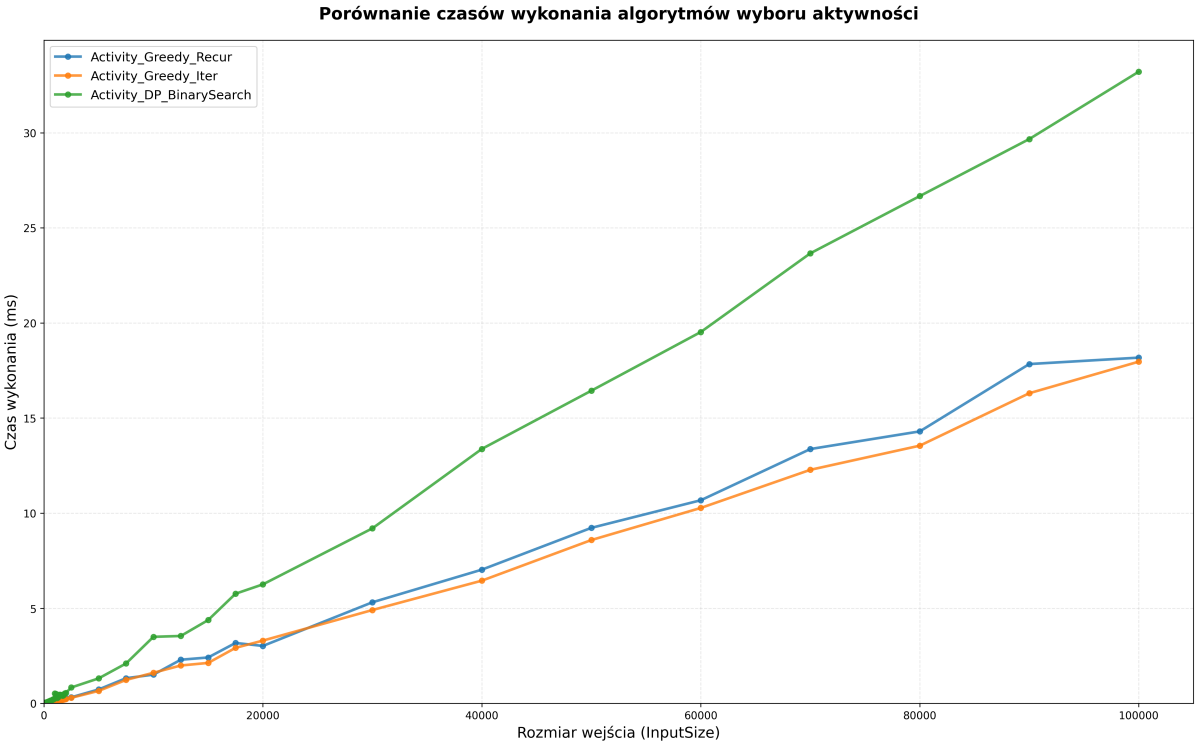


Figure 11: Porównanie różnych podejść do rozwiązania problemu Activity Selector

Przebadano trzy implementacje wyboru maksymalnej liczby niepokrywających się aktywności:

Wnioski z pomiarów:

- **Najwolniejsza jest wersja z binarnym wyszukiwaniem**, mimo że ma podobną złożoność $O(n \log n)$.
- Przyczyny spowolnienia:
  1. Koszt stały związany z wywołaniem `FIND_NEXT` w każdej iteracji, czyli kilkanaście instrukcji dla wyszukiwania binarnego zamiast prostego sprawdzenia w pętli.
  2. Dodatkowe operacje na tablicach: aktualizacja $tab[i]$, $picked[i]$ oraz $next_compatible[i]$, co zwiększa narzut pamięciowy i czasowy.
- Wersje iteracyjna i rekurencyjna działają bardzo szybko, ponieważ dla każdej aktywności sprawdzają tylko **ostatnio wybraną lub poprzednią** aktywność w liniowej pętli.

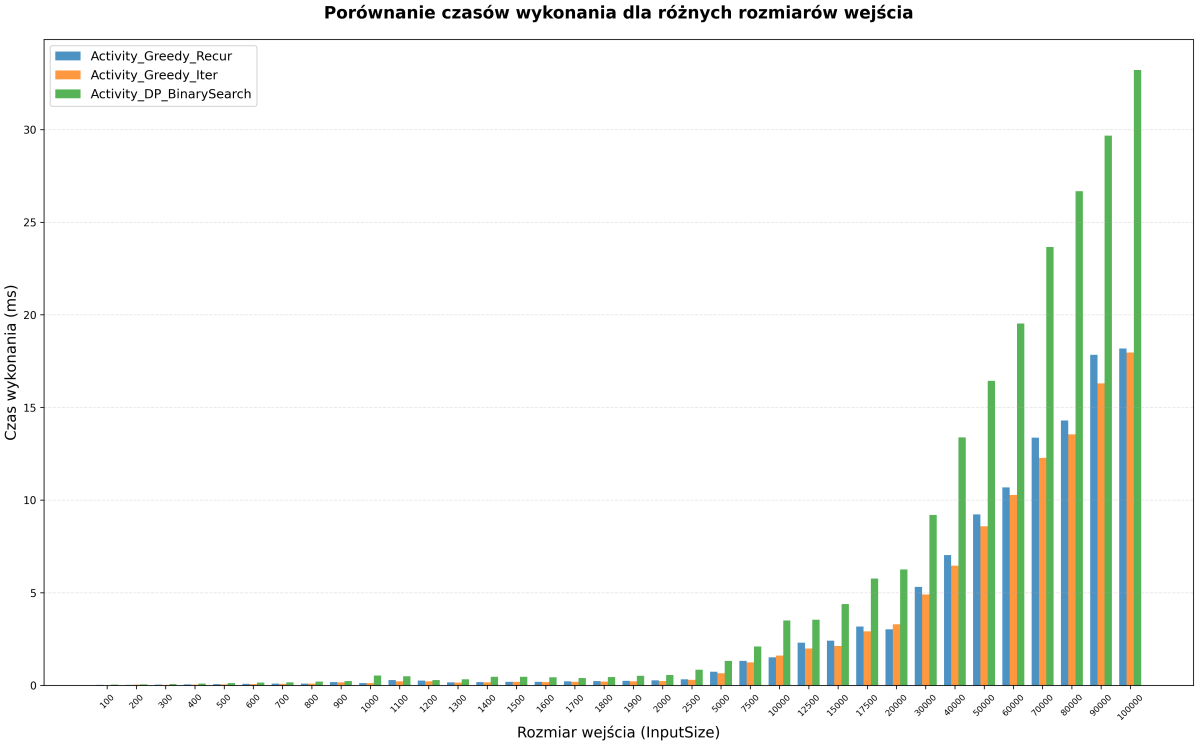


Figure 12: Inna perspektywa stosunku ilości aktywności i czasu dla każdego z trzech algorytmów

4 Kodowanie Huffman

4.1 Algorytm Huffmana

Algorytm Huffmana służy do bezstratnej kompresji danych.

Idea algorytmu:

1. Dla każdego znaku obliczana jest jego częstość występowania.
2. Tworzona jest kolejka priorytetowa (min-heap) z węzłów posortowanych rosnąco według częstości.
3. W każdej iteracji wybierane są dwa węzły o najmniejszych wagach i łączone w nowy węzeł, którego waga jest sumą wag dzieci.
4. Nowy węzeł trafia z powrotem do kolejki.
5. Proces powtarza się aż do powstania jednego drzewa.

Przykładowa tabela znaków:

Znak	Częstość
A	5
B	9
C	12
D	13
E	16
F	45

Proces łączenia:

- Łączymy A(5) i B(9) → węzeł o wadze 14
- Łączymy C(12) i D(13) → węzeł o wadze 25
- Łączymy 14 i E(16) → węzeł o wadze 30
- Łączymy 25 i 30 → węzeł o wadze 55
- Łączymy 45 i 55 → korzeń o wadze 100

Kodowanie: Każde przejście w lewo oznacza bit 0, a w prawo bit 1. Na tej podstawie powstają kody prefiksowe, np.:

Znak	Kod Huffmana
F	0
C	100
D	101
A	1100
B	1101
E	111

4.2 Elementy kodu tenarnej implementacji algorytmu Huffmana

```
void generateCodes(Node* root, map<char, string> &ans, string curr) {
    if (root == nullptr) return;

    if (root->children.empty()) {
        if (root->c != '\0') {
            ans[root->c] = curr;
        }
        return;
    }
    for (int i = 0; i < root->children.size(); i++) {
        //dodajemy 0 1 2 w zaleznosci od dziecka
        generateCodes(root->children[i], ans, curr + to_string(i));
    }
}
```

Figure 13: Generowanie kodów za pomocą DFS

4.2.1 Generowanie kodów za pomocą przeszukiwania DFS

Funkcja `generateCodes` realizuje przejście drzewa Huffmana w głąb (DFS) w celu wygenerowania kodów znaków.

- Każde przejście do kolejnego dziecka oznacza dopisanie cyfry (0, 1 lub 2) do aktualnego kodu.
- Gdy funkcja trafi na liść, zapisuje aktualny ciąg jako kod danego znaku.
